

Quick Path (quick_path.1.6.2604.exe) 仕様書

1. 製品概要

`quick_path` は、ファイルパスやディレクトリパスをリスト形式で管理し、選択したパスに対して迅速にアクション（エクスプローラー起動、ターミナル起動、パスコピー、カスタムコマンド実行）を行うためのコマンドラインインターフェース（CLI）ツールです。

多くの開発プロジェクトやドキュメントディレクトリを抱えるユーザーの作業効率向上を目的としています。

2. 操作方法 (ユーザーマニュアル)

2.1 起動方法

- `quick_path.1.6.2604.exe` を実行します。
- 同一ディレクトリにある `quick_path.ini` を自動的に読み込みます。
- プログラム名が `quick_path.1.6.2604.exe` の場合、最初のピリオドより前の `quick_path` がベースとなります。

2.2 メイン画面の操作

キー	アクション
↑/↓ (矢印)	リスト内の項目を選択（青背景でハイライト）
Enter	選択した項目がフォルダなら階層に入り、ファイルならエクスプローラーで開く
[..]	(フォルダ内) 階層を一つ戻る、または設定メニューに戻る
[e]	Explore: 選択した項目をエクスプローラーで開く
[t]	Terminal: 選択した項目を Windows Terminal で開く
[c]	Copy: 選択したパスをクリップボードにコピーする
[h]	History: 同期コマンドの実行履歴を表示・再コピー、または削除する
[q]	Quit: アプリケーションを終了する
カスタムキー	<code>main.ini</code> で定義されたカスタムアクションを実行します

3. 設定方法 (main.ini)

設定ファイル `main.ini` でパスのリストとカスタムコマンドを定義します。

3.1 [Paths] セクション

表示するパスを `表示名 = パス` の形式で定義します。

```
[Paths]
ソースディレクトリ = D:\Source\Projects
ドキュメント = C:\Users\user\Documents
```

3.2 [Commands] / [SyncCommands] セクション

独自のショートカットキーと実行コマンドを定義します。 `キー = 表示名, コマンド` の形式で記述します。プレースホルダーを使用することで、実行時に内容を動的に変化させることができます。

- `{path}`: 現在選択されているパス（絶対パス）に自動で置換されます。
- `{{任意の名}}`: 実行時にユーザー入力を求めるプロンプトを表示します。
- **[Commands] (非同期実行)**: コマンドをバックグラウンドで起動します。ツールはすぐに次の入力を受け付けます。
- **[SyncCommands] (同期実行)**: コマンドの終了を待ち、その標準出力および標準エラー出力をクリップボードにコピーします。

```
[Commands]
v = VSCode, code {path}
a = Antigravity, antigravity {path}
# 実行時にメッセージ入力を求める例
g = Git-Commit, git commit -m "{{message}}"

[SyncCommands]
s = Show-CSV, powershell -Command "Get-ChildItem"
# 検索ワードを入力して grep する例
f = Find, powershell -Command "Select-String -Pattern '{{keyword}}' -Path '{path}'"
```

- **v** を押すと、選択中のパスが VS Code で開かれます（非同期）。
- **g** を押すと、`{{message}}` の入力を求めるプロンプトが表示され、入力した内容で `git commit` が実行されます。
- **f** を押すと、`{{keyword}}` の入力を求め、`{path}` を対象に検索を実行します。

4. 論理設計 (システム仕様)

4.1 設定読み込みロジック (load_config)

- 実行ファイルのベース名 (`quick_path.1.0.2603.exe` など) をドットで分割した最初の要素 (例: `quick_path`) に基づき、拡張子 `.ini` を付与したファイルを探します。
- `config.optionxform = str` を設定しているため、`[Paths]` セクションなどのキーは大文字小文字が保持された状態で表示されます。
- `utf-8-sig` エンコーディングで読み込むため、BOM付きの日本語INIファイルにも対応しています。

4.2 実行制御ロジック (run_action / 階層移動)

- **Enterキーの挙動:**
 - 選択中の項目がディレクトリの場合、その階層に移動し、内容を表示します。
 - 選択中の項目が [`..`] の場合、一つ前の表示状態（親ディレクトリまたは初期設定メニュー）に戻ります。
 - 選択中の項目がファイルの場合、標準アクションの **Explore ([e])** を実行します。
- **ディレクトリ探索初期化:**
 - `os.scandir` を使用してディレクトリ内の項目を一覧化し、ディレクトリを優先してソート表示します。
- **各アクション実行:**
 - 選択されたパスがファイルを指している場合はその親ディレクトリ、フォルダを指している場合はそのパス自体をアクションの実行対象（作業ディレクトリ）と見なします。
- **Terminal ([t]):** `start wt -d "{dir_path}"` コマンドにより独立した Windows Terminal プロセスを起動します。
- **非同期コマンド ([Commands]):**
 - `os.system(f'start "{final_cmd}")` を使用して実行されます。
 - プロセスは独立して動作し、ツール側の操作を妨げません。
- **同期コマンド ([SyncCommands]):**
 - `subprocess.run` を使用して実行され、終了までツールが待機します。
 - 実行結果（標準出力 + 標準エラー）は PowerShell の `Set-Clipboard` を介してクリップボードに自動的にコピーされます。
 - **履歴保存:** 実行日時、ラベル、コマンドライン、および出力結果が自動的に SQLite データベース (`quick_path.db`) に保存されます。
- **プレースホルダー置換 (resolve_placeholders):**
 - `{path}` プレースホルダーは、ダブルクォートで囲まれた絶対パスに置換されます（既存機能）。
 - `{{name}}` 形式のプレースホルダーがある場合、実行前にコンソールを介してユーザーに入力を促します。
 - 同一の指定が複数含まれる場合は、1回の入力ですべてが置換されます。
 - パラメタの入力が `Ctrl+C` 等で中断された場合、コマンドの実行全体をキャンセルします。

4.3 履歴管理ロジック (show_history)

- **データベース接続:**
 - 実行ファイルと同一ディレクトリの `quick_path.db` に接続します。
 - `sync_history` テーブルが存在しない場合は起動時に自動生成されます。
- **履歴表示機能 ([h]):**
 - 直近 100 件の同期コマンド履歴を降順（新しい順）でリスト表示します。
 - 履歴一覧から項目を選択して `c` を押すと、その時の出力結果が再度クリップボードにコピーされます。
 - 履歴一覧から項目を選択して `f` を押すと、ファイル名を入力してその内容を ANSI (CP932) エンコードでファイル出力（実行ファイルと同一フォルダ内）できます。
 - 履歴一覧から項目を選択して `d` を押すと、確認メッセージを表示した後、その履歴レコードをデータベースから物理削除します。
 - `q` または `Esc` キーでメインメニューに戻ります。

4.4 UI表示ロジック (メニュー部の表示)

- **コマンドキーヘルプの表示:**
 - 画面上部のコマンドキーヘルプは、視認性向上のため以下のルールで改行して表示されます。
 - **第1行 (内蔵系コマンド):** 全ての内蔵コマンド (`[Enter]`, `[e]`, `[t]`, `[c]`, `[h]`, `[q]`) を 1 行にまとめて表示した後に改行します。
 - **第2行以降 (ユーザー定義コマンド):** `[Commands]`, `[SyncCommands]` セクションの内容を、1行 76 文字を目安に、コマンド単位で適切に改行を挿入して表示します。

5. 物理設計 (ファイル構成・ビルド)

5.1 ファイル構成

- `main.py`: プログラム本体
- `main.ini`: ユーザー設定ファイル
- `quick_path.db`: 実行履歴データベース (自動生成)
- `assets/ken2tec.ico`: 実行ファイルのアイコン
- `build.ps1`: ビルド用PowerShellスクリプト

5.2 ビルド環境

- **Python**: 3.x
- **環境**: `dev` という名称の Conda 仮想環境でのビルドを想定しています。
- **主要モジュール**: `pyinstaller`, `configparser`, `msvcrt`, `re`, `subprocess`, `sqlite3`
- **ビルドコマンド**:

```
# build.ps1 を実行
.\build.ps1
```

内部的には PyInstaller を使用し、依存関係を1つの実行ファイル (`--onefile`) にパッケージングします。不要なライブラリ (`pandas`, `numpy`等) はサイズ削減のため明示的に除外されます。

6. 注意点

- **Windows Terminal**: `[t]` アクションは Windows Terminal (`wt`) がインストールされている必要があります。
- **パスのクォート**: カスタムコマンド内で `{path}` を使用する場合、内部で自動的にクォート処理が行われます。

7. 更新履歴

- **1.6.2604 (2026-04-10)**:
 - ヒストリーモード (`[h]`) において、選択したレコードを削除する機能 (`[d]`) を追加。削除前の確認フェーズを実装。
- **1.5.2604 (2026-04-07)**:
 - ヒストリーモード (`[h]`) の表示において、25件を超える履歴がある場合のスクロール表示に対応。
- **1.4.2604 (2026-04-06)**:
 - コマンドキーヘルプの表示を整理。
 - 内蔵コマンド (`Enter`, `e`, `t`, `c`, `h`, `q`) を一行目に、ユーザー定義コマンドを二行目以降 (76文字制限) に表示するよう変更。
 - 仕様書のPDF出力時の視認性を調整。
- **1.3.2604 (2026-04-04)**:
 - ユーザー定義パラメータ `{{name}}` に対応。